Universidade do Vale do Itajaí
Escola Politécnica
Curso de Ciência da Computação
Algoritmos e Programação II (22817)

Definição do trabalho da M2 – LABIRINTO GIRATÓRIO: A EVOLUÇÃO

Data da entrega: ___/___/_____
Data da defesa: ___/___/_____
Modalidade: em TRIO (a mesma equipe da M1).

Visão Geral:

O trabalho da M2 consiste na evolução do jogo Labirinto Giratório desenvolvido na M1. As regras do jogo não mudam: o que muda é a qualidade da construção do programa e aquilo que ele é capaz de guardar e de ler. Na M1, o estado da partida circulava entre as funções como uma fila de parâmetros soltos (matriz, tamanho, orientação, posição do jogador, contadores), os mapas estavam escritos dentro do código, a opção "Continuar" só funcionava enquanto o programa estava aberto e as sequências de teclas que resolviam os mapas existiam apenas no papel.

Agora que a turma conhece structs e operações com arquivos, a M2 resolve essas limitações: (1) todo o estado da partida passa a ser representado por structs; (2) os mapas passam a ser lidos de arquivos, em um formato padronizado, e validados durante a leitura; (3) a sequência de teclas que resolve cada mapa também vira um arquivo, e o próprio jogo passa a reproduzi-la para comprovar que o mapa tem solução; (4) o "Continuar" passa a sobreviver ao fechamento do programa; (5) as vitórias passam a ser registradas em um ranking.

Ponto de partida: o código da própria equipe na M1. Não se trata de um trabalho novo, mas de uma refatoração com novas funcionalidades — e a evolução em relação à M1 faz parte da avaliação.

O que muda em relação à M1:

Aspecto	Na M1	Na M2
Estado da partida	Variáveis soltas passadas como parâmetros (struct proibida)	Struct Jogo, com struct aninhada e matriz dentro dela (struct obrigatória)
Mapas	Matrizes escritas no código (mínimo 3)	Arquivos .csv lidos e validados em tempo de execução (mínimo 5)
Solução dos mapas	Sequência de teclas entregue no papel	Arquivo .sol que o próprio jogo reproduz e verifica
Continuar	Só em memória, enquanto o programa está aberto	Arquivo de jogo salvo, que sobrevive ao fechamento do programa
Histórico	Não existe	Ranking em arquivo, ordenado, com os melhores resultados de cada mapa
Menu	Novo jogo, Continuar, Sobre, Fim	Novo jogo, Continuar, Ver solução, Recordes, Sobre, Fim

Regras do jogo:

Todas as regras definidas na M1 continuam valendo, sem nenhuma alteração: movimentação, rotação, gravidade, portas, esmagamento, vitória, derrota, teclas R e ESC e informações exibidas na tela. Os defeitos que a correção da M1 tenha apontado devem ser corrigidos agora, e a equipe deve listá-los no README (ver Entrega).

Estruturas de dados (structs):

Na M1 era proibido usar struct. Na M2 isso se inverte: todo o estado de uma partida deve viajar entre as funções dentro de uma struct, e não mais como uma lista de parâmetros soltos. O mínimo esperado é:

```
const int TAM = 21;              // dimensão máxima do cenário

struct Posicao {
    int lin, col;
};

struct Jogo {
    int cenario[TAM][TAM];       // só as n x n primeiras posições são usadas
    int n;                       // dimensão do mapa atual (de 11 até TAM)
    int orientacao;              // 0, 1, 2 ou 3 (múltiplos de 90°)
    Posicao jogador;
    int movimentos, rotacoes;
    int numeroMapa;
    string titulo;
};

struct Recorde {
    string nome;
    int mapa, movimentos, rotacoes;
};
```

- A equipe pode acrescentar campos e criar outras structs, mas não deve manter variáveis soltas que representem o estado da partida.
- Funções que só consultam um valor simples (por exemplo, `portaEstaFechada(int celula, int orientacao)`) continuam recebendo apenas o que precisam. Não passe a struct inteira onde um `int` basta.
- Continua proibido o uso de variáveis globais. Constantes, como `TAM`, são permitidas.
- O cenário não tem mais tamanho fixo em 11 x 11: cada mapa tem a sua dimensão `n`, de 11 até `TAM`. O código não pode assumir que `n` é sempre 11.

Exemplos de assinatura, apenas para ilustrar a forma esperada (compare com as da M1, como moveJogador com seis parâmetros: a diferença deve ficar evidente):

```
void aplicaGravidade(Jogo &j);
void giraDireita(Jogo &j);
void moveJogador(Jogo &j, char tecla);
int  processaTecla(Jogo &j, char tecla);
bool mapaEstavel(Jogo j);
template <typename T> void trocar(T &a, T &b);
```

Arquivos do jogo:

O jogo trabalha com quatro tipos de arquivo. Todos ficam na pasta de execução do programa, sem subpastas, e o jogo deve ser executado a partir dessa pasta.

Arquivo	Conteúdo	Formato	Escrito por
`mapaN.csv`	Cenário inicial do mapa N (N = 1, 2, 3, …)	Padronizado (ver abaixo)	A equipe, à mão
`mapaN.sol`	Sequência de teclas que resolve o mapa N	Padronizado (ver abaixo)	A equipe, à mão
`jogo_salvo.csv`	Partida interrompida pelo jogador	Livre, descrito no README	O jogo
`recordes.csv`	Vitórias registradas	Padronizado (ver abaixo)	O jogo

Os formatos "padronizados" são iguais para todas as equipes: no dia da defesa, o jogo será testado com mapas escritos pelo professor.

Mapas em arquivo:

O jogo não pode ter um número fixo de mapas escrito no código. Ele deve descobrir os mapas existentes tentando abrir `mapa1.csv`, `mapa2.csv`, `mapa3.csv` e assim por diante, até que um deles não exista. Cada arquivo de mapa tem o seguinte formato:

- linha 1: a dimensão `n` do mapa;
- linha 2: o título do mapa (texto livre, sem vírgulas), exibido na escolha do mapa;
- linhas 3 a `n+2`: as `n` linhas do cenário, cada uma com `n` códigos de 0 a 7 (os mesmos da tabela da M1) separados por vírgula, sem espaços.

O código `2` marca apenas a posição inicial do jogador, que sempre começa sobre uma célula vazia: em arquivo, o jogador não pode começar sobre a alavanca. Exemplo completo de arquivo (`mapa_exemplo.csv`):

```
11
Primeiro giro
1,1,1,1,1,1,1,1,1,1,1
1,2,0,0,0,0,1,0,0,0,1
1,0,0,0,0,0,1,0,0,0,1
1,0,0,4,0,0,1,0,0,0,1
1,0,0,0,0,0,1,0,0,0,1
1,0,0,0,0,0,3,0,0,0,1
1,0,0,0,0,0,1,0,0,0,1
1,0,0,0,0,0,1,0,0,0,1
1,0,0,0,0,0,1,0,5,0,1
1,0,0,0,0,0,1,0,0,0,1
1,1,1,1,1,1,1,1,1,1,1
```

Neste mapa, o bloco (3) fecha a única passagem da parede central. Girando o cenário para a direita (E) sobre a alavanca (4), a parede central passa a ser um "teto" com a passagem tomada pelo bloco; a gravidade derruba o bloco, a passagem se abre e o caminho até a saída (5) fica livre. Sem girar, o mapa não tem solução.

Validação na leitura

Ao carregar um mapa, o jogo deve verificar o arquivo e recusá-lo, informando o motivo, nos casos abaixo. Um mapa recusado aparece na escolha de mapas como indisponível, com o motivo, e nunca derruba o programa:

- dimensão ausente, não numérica ou fora do intervalo de 11 até TAM;
- título ausente, ou menos linhas do que o informado, ou linha com quantidade de colunas diferente de n;
- valor não numérico ou fora do intervalo de 0 a 7;
- quantidade de jogadores diferente de 1, ou ausência de saída;
- mapa instável: algum bloco começa no ar. A verificação deve reutilizar a função que aplica a gravidade (aplique a gravidade sobre uma cópia do estado inicial e compare com o original).

Dicas sobre arquivos

- `stoi(texto)` converte um texto em `int`, da mesma forma que `stof` converte em `float`.
- Para separar uma linha nos seus valores, use um `stringstream` junto com `getline(seu_stringstream, variavel, ',')`: o terceiro parâmetro faz a leitura parar na vírgula.
- Para saber se um arquivo existe, tente abri-lo e teste `is_open()`. `ios_base::app` acrescenta ao final do arquivo sem apagá-lo, e `remove("nome")` (biblioteca `<cstdio>`) apaga um arquivo.
- Uma função que acessa arquivo não deve imprimir na tela. Para informar o motivo de uma falha, devolva um bool e preencha um parâmetro string por referência com a mensagem.

Solução em arquivo e a opção "Ver solução":

Cada mapa `mapaN.csv` deve ter um arquivo `mapaN.sol` com uma única linha contendo as teclas que o resolvem, usando apenas W, A, S, D, Q e E (maiúsculas ou minúsculas), sem espaços. A solução do mapa de exemplo (`mapa_exemplo.sol`) é:

```
ssddEaasssssaaa
```

Na opção "Ver solução", o jogador escolhe um mapa e o jogo executa as teclas do arquivo uma a uma, desenhando o cenário após cada tecla (com uma pequena pausa ou a cada tecla pressionada). Ao final, o jogo informa o resultado:

- solução válida: o jogador chegou à saída (exibir o total de teclas, movimentos e rotações);
- as teclas acabaram e o jogador não chegou à saída;
- derrota durante a reprodução, informando em qual tecla ocorreu;
- tecla inválida no arquivo, informando a posição dela;
- arquivo .sol inexistente para o mapa escolhido.

Essa opção deve oferecer também "Verificar todos os mapas": executa, sem desenhar nada, a solução de cada mapa e exibe uma lista com o número do mapa, o título e o resultado. Reproduzir uma solução e jogar com o teclado devem usar **exatamente a mesma função de tratamento de teclas**: nenhuma regra do jogo pode ser escrita duas vezes. Uma solução reproduzida não vale para o ranking.

Continuar persistente:

A opção "Continuar" deixa de depender da memória do programa. Sempre que a partida for interrompida com ESC, o jogo deve gravar a partida em jogo_salvo.csv. Ao abrir o programa novamente, "Continuar"

deve estar disponível e retomar exatamente o que foi deixado: o mapa, o cenário no estado atual, a orientação, a posição do jogador (inclusive sobre a alavanca) e os contadores.

O formato desse arquivo é livre — afinal, ele depende da forma como a equipe resolveu, na M1, a representação de duas coisas na mesma célula —, mas deve ser descrito no README. Além disso:

- "Continuar" só fica disponível se existir um jogo salvo válido; um arquivo ausente, vazio ou corrompido deve ser tratado como "sem jogo salvo", sem travar o programa;
- se existir jogo salvo e o jogador escolher "Novo jogo", o programa deve pedir confirmação antes de substituí-lo;
- ao vencer a fase, o jogo salvo daquela partida deve ser invalidado.

Ranking de recordes:

Ao vencer uma fase, o jogo pede o nome do jogador (até 15 caracteres, sem vírgulas) e acrescenta uma linha ao final de `recordes.csv`, no formato `nome,mapa,movimentos,rotacoes`. Exemplo:

```
Ana,1,14,1
Bruno,1,22,3
Ana,2,41,5
```

A opção "Recordes" pergunta o mapa, lê o arquivo para um vetor de Recorde (no máximo 200 registros) e exibe os 5 melhores resultados daquele mapa, ordenados por menor número de movimentos; em caso de empate, por menor número de rotações. Se não houver registros, exibe uma mensagem amigável; se o arquivo não existir, ele é criado na primeira vitória.

A ordenação pode usar o algoritmo que a equipe preferir, mas a troca de dois registros deve ser feita pela função template `trocar` (a mesma ideia dos exercícios de structs), chamada com elementos do tipo `Recorde`.

Menu:

O jogo deverá possuir um menu com as opções abaixo. Como na M1, o jogo só deve ser encerrado pela opção "Fim"; qualquer outra opção retorna ao menu ao fim de sua execução, e deve ser possível voltar ao menu a qualquer momento.

Opção	Exigências
Novo jogo	Lista os mapas descobertos pelo título (os inválidos aparecem como indisponíveis, com o motivo). O jogador escolhe um mapa específico ou um mapa aleatório entre os válidos. Se houver jogo salvo, pede confirmação antes de substituí-lo.
Continuar	Lê o jogo salvo em arquivo e retoma a partida exatamente como foi deixada. Disponível somente se houver jogo salvo válido.
Ver solução	Reproduz a solução de um mapa escolhido e oferece "Verificar todos os mapas" (ver seção própria).
Recordes	Exibe os 5 melhores resultados do mapa escolhido, a partir de recordes.csv.
Sobre	Exibe a equipe (nome de cada membro), o mês/ano e o nome do professor/disciplina. Explica as regras do jogo, a função de cada tecla e as novas opções do menu.

Opção	Exigências
Fim	Encerra o programa. É a única opção que o encerra.

Regras para o desenvolvimento:

- O jogo continua sendo desenvolvido em C/C++, sem motores de jogos, a partir do código da equipe na M1 e dos códigos de desenho e movimentação fornecidos pelo professor (versão Linux ou Windows, a mesma da M1 ou outra, desde que informada).
- Não é permitida a utilização de variáveis globais.
- Recursos permitidos e esperados: funções, passagem de parâmetros por valor e por referência, vetores e matrizes, structs, arquivos (fstream), stringstream e funções template. Não utilizar classes, ponteiros nem vector (utilize vetores simples).
- A função main deve ser curta: essencialmente o menu e o laço principal chamando sub-rotinas. Nenhuma regra do jogo, nenhum acesso a arquivo e nenhum desenho de tela devem estar escritos dentro dela.
- Cada sub-rotina deve cuidar de **uma só** das três responsabilidades: aplicar as regras (altera o `Jogo`, não imprime e não acessa arquivo), desenhar a tela (imprime, não altera) ou acessar arquivos (lê e grava, não imprime e não aplica regras).
- Os arquivos devem ser tratados com segurança: arquivo inexistente, vazio ou com conteúdo inválido nunca pode travar nem encerrar o programa de forma inesperada.
- Produzam todo o código pensando na possibilidade de que novas funcionalidades poderão ser solicitadas no futuro.

Mapas da M2:

- O jogo deverá possuir no mínimo 5 mapas em arquivo: os 3 mapas da M1, migrados para o formato novo, e pelo menos 2 mapas novos, independentemente do número de integrantes.
- Os mapas novos são autorais e deverão ser submetidos ao professor antes da implementação, que garantirá que nenhuma equipe implemente um mapa semelhante ao de outra. Entre eles deve haver pelo menos um mapa com dimensão de 15 x 15 ou mais, e pelo menos um que exija três ou mais rotações.
- Todo mapa deve ter o seu arquivo .sol, e o jogo deve comprovar, pela opção "Verificar todos os mapas", que todas as soluções funcionam. Um mapa cuja solução não passe na verificação não será considerado.
- As exigências de variedade da M1 continuam valendo para o conjunto de mapas (queda de bloco que abre passagem, porta que some com a rotação e duas ou mais rotações). O estado inicial de todo mapa deve estar estável.

Sobre o uso de sub-rotinas:

Continua sendo o principal critério técnico do trabalho. A decomposição pedida na M1 continua valendo, agora trabalhando sobre a struct Jogo. Além dela, espera-se no mínimo sub-rotinas para:

- carregar um mapa a partir do seu número, devolvendo o motivo quando o arquivo for inválido;
- verificar se um mapa é válido e se está estável;
- descobrir quantos mapas existem;
- tratar uma tecla (a mesma função para o teclado e para a reprodução de soluções);

- executar uma solução, sem desenhar, devolvendo o resultado, e reproduzi-la com desenho;
- salvar o jogo, carregar o jogo salvo e informar se existe jogo salvo válido;
- registrar um recorde, carregar os recordes, ordenar e exibir o ranking;
- a função template trocar.

Desafios de projeto:

Os desafios de projeto da M1 continuam valendo, e as decisões tomadas lá devem ser revistas à luz das structs. Há novos pontos sem resposta única, que fazem parte da avaliação e devem ser decididos e justificados na apresentação:

- A struct deve ser passada por valor ou por referência? Em quais funções cada forma é adequada? Quanto custa copiar uma struct que carrega uma matriz inteira, e em que situações essa cópia é exatamente o que se deseja?
- O campo com a posição do jogador é redundante com o conteúdo da matriz? Guardar a posição ou localizá-la a cada uso: qual é o risco de inconsistência?
- O que precisa ser gravado no jogo salvo e o que pode ser recalculado? Quando gravar: a cada movimento, a cada rotação ou só ao sair?
- Como garantir que jogar e reproduzir uma solução usam exatamente as mesmas regras, sem código duplicado?
- Qual critério de desempate usar no ranking, e por quê?

Desafios extras (opcionais):

Quem concluir todo o conteúdo obrigatório pode implementar um ou mais dos desafios abaixo. Cada desafio concluído e apresentado vale até 5% adicionais na nota do trabalho, limitada ao máximo de 100%:

- **Dica automática:** uma tecla (por exemplo, H) mostra a próxima tecla de uma solução a partir do estado atual da partida, calculada por uma função recursiva que experimenta as teclas sobre cópias do estado (passagem da struct por valor), com limite de profundidade e controle de estados repetidos.
- **Editor de mapas:** uma opção do menu que permite desenhar um mapa na tela e gravá-lo no formato padronizado, validando-o antes de gravar.
- **Gravação de partida:** ao vencer, gravar as teclas pressionadas em um arquivo .sol, que pode ser reproduzido pela mesma opção “Ver solução”.

ENTREGA:

- Postar um arquivo texto com o link para o trabalho no GitHub. Deve ser o mesmo repositório da M1, com a tag `m1` marcando o commit entregue naquela etapa e a evolução registrada em commits posteriores, de modo que o histórico permita comparar as duas versões.

O repositório deve conter, além do código:

  - **README.md** com: (a) a versão em que o trabalho foi desenvolvido (Linux ou Windows); (b) a descrição do formato do arquivo de jogo salvo; (c) uma tabela antes/depois com pelo menos 5 funções da M1 (assinatura na M1, assinatura na M2 e o motivo da mudança); (d) a lista dos defeitos da M1 que foram corrigidos; (e) as decisões de projeto; (f) os desafios extras implementados, se houver.

– Os arquivos `mapaN.csv` e `mapaN.sol` de todos os mapas (mínimo 5).

APRESENTAÇÃO:

- A apresentação e a nota da apresentação serão individuais; além disso, todas as notas relativas ao código dependem do desempenho na apresentação. Sem apresentação, o trabalho terá nota ZERO.
- No dia da defesa, o professor executará o jogo da equipe com: (a) um mapa válido de autoria dele, acompanhado do respectivo .sol; (b) um arquivo de mapa inválido; (c) sem os arquivos jogo_salvo.csv e recordes.csv. O jogo não pode travar em nenhum desses casos.
- O professor sorteará uma função da M1 e pedirá ao aluno que mostre, no histórico do repositório, como ela ficou na M2 e por que mudou.
- O aluno deverá explicar e justificar as decisões dos desafios de projeto.

Critérios de Avaliação:

1. **Organização e clareza do código = 10% da nota** (inclui o README e a separação entre regras, tela e arquivos).
2. **Funcionamento correto conforme a especificação = 20% da nota** (inclui a robustez diante de arquivos ausentes ou inválidos e o funcionamento com os mapas do professor).
3. **Recursos da linguagem utilizados, com destaque para structs, arquivos e passagem de parâmetros = 20% da nota**.
4. **Apresentação do código = 50% da nota e é individual** (inclui a comparação entre a M1 e a M2).

Obs1: Cópia de soluções de colegas ou da internet acarretarão nota 0 para todos os envolvidos.

Obs2: Trabalhos entregues e/ou apresentados em atraso receberão um desconto inicial de 20% na nota final, além de um desconto progressivo de 10% por aula. Ou seja, apresentar na aula seguinte à data estipulada deixa a nota máxima do trabalho em 70%.